

For my project, I wanted to analyze a student dataset and find correlations between student performance and unsuspecting factors, such as family income and parental education. Those are things that may intuitively be correlated, but I wondered if it was actually true, if there were external factors outside of students' control affecting their performance in school. I construct a graph, perform graph manipulation, and BFS. My biggest challenge was constructing the graph and making sure headers were correctly read..

After loading my dataset into Rust, the first thing I had to do was some data cleaning. This dataset has about 5,000 entries (students), so there was a lot to look through, but I did a few sweeps and it became clear that there was some missing data. I wasn't sure if there were duplicates or not, but I made sure to address that in my data cleaning. The first two images below are the code I used to clean my data that once existed in main.rs. I made sure the new CSV file contained the same column labels and that they were typed correctly. In the main function, I skip rows with missing or corrupt data and remove duplicates if any. When I ran my main.rs, a new CSV file which I used for the rest of my project, the cleaned version of my original dataset, was created. I looked through it and it looked complete.

```
use csv::{ReaderBuilder, WriterBuilder};
use serde::{Deserialize, Serialize};
use std::collections::HashSet;
use std::error::Error;
use std::fs::File;

#[derive(Debug, Deserialize, Serialize, Clone, PartialEq, Eq, Hash)]
struct Student {
    Student_ID: String,
    First_Name: String,
    Last_Name: String,
    Email: String,
    Gender: String,
    Age: u8,
    Department: String,
    #[serde(rename = "Attendance (%)")]
    Attendance: f32,
    Midterm_Score: f32,
    Final_Score: f32,
    Assignments_Avg: f32,
    Quizzes_Avg: f32,
    #[serde(rename = "Participation_Score")]
    Participation_Score: f32, // fixed typo
    Projects_Score: f32,
    Total_Score: f32,
    Grade: String,
    Study_Hours_per_Week: f32,
    Extracurricular_Activities: String,
    Internet_Access_at_Home: String,
    Parent_Education_Level: String,
    Family_Income_Level: String,
    #[serde(rename = "Stress_Level (1-10)")]
    Stress_Level: u8,
    Sleep_Hours_per_Night: f32,
}

fn main() -> Result<(), Box<dyn Error>> {
    let input_path = "Students_Grading_Dataset.csv";
    let output_path = "Cleaned_Students_Grading_Dataset.csv";

    let mut reader = ReaderBuilder::new()
        .trim(csv::Trim::All)
        .from_path(input_path)?;
    let mut writer = WriterBuilder::new().from_path(output_path)?;

    let mut seen = HashSet::new();

    for result in reader.deserialize() {
        let record: Student = match result {
            Ok(r) => r,
            // skip rows with missing or corrupt data
            Err(_) => continue,
        };

        if record.Stress_Level > 10
            || record.Sleep_Hours_per_Night <= 0.0
            || record.Attendance < 50.0
        {
            continue;
        }
        // remove duplicates
        if seen.contains(&record) {
            continue;
        }

        seen.insert(record.clone());
        writer.serialize(record)?;
    }

    writer.flush()?;
    Ok(())
}
```

Once I had a cleaned CSV file to use for graph algorithms, the first thing to do was construct a graph, but to do so, I had to create Student instances. As a side note, all of my code was in main.rs and totaled to be around 200 lines of code, but I've split them up into modules. In student.rs, I created a Student struct based on the elements I wanted to analyze, which included their ID to identify them, their total score and attendance, and how that may have been related to parental education, family income, and stress level. Again, I used parental education and family income because those were the attributes I suspected may have been correlated, but truly wasn't sure, whereas the stress level I knew had to have been correlated. I then implement the Student struct with a constructor that creates a new Student instance with provided values. I initialize the fields to be filled according to the student themselves.

Now that the creation of a Student instance has been initialized, it was time to actually construct the graph. I import the Student struct, which is used to represent a student, as well as the Error trait and

csv crate for reading csv files. The first function in this file loads student data from the provided CSV file and returns a tuple containing both a vector of Student structs, and a vector of column headers from the CSV file. Within the function, it creates a CSV reader for the provided file path and initializes empty vectors to hold student data and CSV headers. It attempts to read the CSV headers, and if successful it stores the headers as strings in the headers vector. If reading the header fails, a Boxed Error is returned. Then, still in the function, there is a for loop to loop through each record (row) in the CSV file. If reading a record succeeded, it is parsed into a student. Each value from the CSV record is extracted and converted to their respective types to create a new Student instance using the parsed data. The newly created student is pushed into the students vector.

The final module I have is the graph_utils.rs module, which I use to perform graph manipulation. I first import necessary types for graph manipulation and other utilities, including petgraph and the Student struct. I have two functions in this module. The first computes the average distance between a random sample of nodes in the graph. It takes in a graph to perform the distance computation on, as well as a group of nodes (students) to randomly sample. I randomly selected a random sample of nodes from the group and initialized the accumulated total distance between sampled nodes, as well as a counter of nodes to 0. I then perform BFS from each source node in the sample. Visited nodes and their distances are tracked, and there is a queue for BFS. We start with the source node and then perform BFS to compute distances. For each target node in the sample, we compute the distance to the source, add distance to the total, and increment the valid distance count by one. The function returns the average distance, or 0 if no distances were computed. The second function calculates the mean of a particular attribute for the students in the provided group. It takes in a list of nodes to calculate the mean for, the graph of students, and a function to select the attribute from a student. I also initialized the selector function type to be a numerical floating point. The function sums up the selected attribute for all students in the group, applies the selector function to each student, and returns the mean by dividing the total by the number of nodes.

Finally, in my main.rs, I import the necessary modules and functions, as well as the functions for graph computations, loading students from a CSV file, graph structures from the petgraph library, and error handling imports. In my main function, I first load student data from the cleaned for graph students CSV file I have. The function returns a tuple of students and headers. It then creates an undirected graph to represent the student relationships with vectors to hold student nodes and categorize them into high or low performers. In a for loop, we iterate through each student, adding them as nodes to the graph and categorize them based on their score. We add the student as a node, store the node for later use, and categorize them as a high performer if their score is greater than or equal to 60, and a low performer otherwise. We then add edges between adjacent student nodes, forming a linear structure in the graph. I print the graph construction details, initially for debugging purposes, but now just for informative purposes. Then we compute and print the average distances for high and lower performers from 100 samples to take for distance calculations. We calculate and print the mean values for the various student features that I chose earlier, and output those calculated mean values for each feature.

I have my unit tests for the functions used in the program at the end of main.rs. I import everything from the main module to use in my tests. The first test tests the calculate_mean function. I create some sample students with different attributes, create a graph, and add the students as nodes. We calculate the mean attendance of the students in the nodes and answer that the calculated mean is as expected. I also tested the compute_avg_distance_sample function, where I again create some sample students, create a graph and add the students as nodes, connect the nodes with edges and create a vector of

nodes to calculate the average distance sample. I set the sample size to 2 for distance calculation, compute the average distance for the given nodes and sample size, and assert that the average distance is greater than 0, which is a reasonable check.

Looking at my output for cargo run, we see that I get confirmation that graph construction was complete with all 5,000 student nodes and 4,999 edges. This tells me that my graph was successfully constructed and the graph might be a spanning tree or close to one, meaning that it's minimally connected with no cycles. There are way more high performers (4,017) compared to low performers (983), and a large majority (about 80%) of students are categorized as high performers. The sampled average distances tells us the average path length between nodes (students) swamped within each group. Low performers are, on average, closer to each other in the network than high performers. This could suggest denser or tighter subgraphs among low performers, or possibly that low performers are more socially clustered. For attribute analysis, both groups have the same attendance mean of 75.36%, meaning that attendance wouldn't really explain the performance gap in this dataset. When it comes to the parent education mean, there is a very slight difference, so parental education doesn't seem to explain the gap either. There is a small difference for the family income mean between high and low performers. Slightly higher income levels are associated with higher performance, but this difference may not be meaningful. The most relevant finding is what I sort of intuitively assumed from the beginning, which is that low performers report slightly higher stress levels, though the difference is still modest. Looking at my output for cargo test, you can see that both of my tests run and pass.

While I didn't discover what I was looking for, it's good to know that parental education level and family income don't differentiate high performers from low performers, meaning it is more in students' control how well they do in school.

```
Graph construction complete.  
Number of student nodes: 5000  
Number of edges: 4999  
High performers count: 4017  
Low performers count: 983  
Sampled average distance (high performers): 1497.74  
Sampled average distance (low performers): 1750.85  
High Performers - Attendance Mean: 75.36  
Low Performers - Attendance Mean: 75.36  
High Performers - Parent Education: Mean 2.01  
Low Performers - Parent Education: Mean 2.00  
High Performers - Family Income: Mean 1.00  
Low Performers - Family Income: Mean 0.95  
High Performers - Stress Level: Mean 5.48  
Low Performers - Stress Level: Mean 5.63
```

```
running 2 tests  
test tests::test_calculate_mean ... ok  
test tests::test_compute_avg_distance_sample ... ok  
  
test result: ok. 2 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out; finished in 0.00s
```